

# **INDOOR POSITION DETECTION SYSTEM**

A WiFi RSSI-Based Android Application for Real-Time Indoor Localization

By

**A S M Mehedi Hassan (Exam Roll: 220545)**  
**Jubayer Bishwas Shadin(Exam Roll:220549 )**  
**Hrishikesh Kanu Bintu (Exam Roll: 220554)**

A project report submitted to the Institute of Information Technology in partial fulfilment  
of the requirements for the degree of B.Sc. in ICT

**Supervisor: Dr. Shamim Al Mamun**  
**Professor, IIT, JU**



Institute of Information Technology  
Jahangirnagar University  
Savar, Dhaka-1342, Bangladesh

June 29,2026

# **DECLARATION**

This project report is submitted to the Institute of Information Technology, Jahangirnagar University, Savar, Dhaka in partial fulfilment of the requirements for the B.Sc. (Hons.) degree in ICT. This is to certify that this project work, titled “Indoor Position Detection System,” has been developed as part of the academic curriculum under the relevant Project Work and Course Viva course.

We hereby declare that this report is based on the results obtained through our own work and implementation. All materials, code snippets, or research findings sourced from other authors or researchers have been duly acknowledged and referenced. This report, neither in whole nor in part, has been previously submitted to any other university or institute for any degree or diploma.

## **CANDIDATE SIGNATURE**

---

A S M Mehedi Hassan

Roll:220545

---

Jubayer Bishwas Shadin

Roll:220549

---

Hrishikesh Kanu Bintu

Roll:220554

## **CERTIFICATE**

The project titled “Indoor Position Detection System” submitted by A S M Mehedi Hassan (Exam Roll:220545), Jubayer Bishwas Shadin (Exam Roll:220549), Hrishikesh Kanu Bintu (Exam Roll: 220554) has been accepted as satisfactory in partial fulfilment of the requirement for the degree of Bachelor of Science in Information Technology on [Submission Date].

---

Dr. Shamim Al Mamun  
Professor, Institute of Information Technology, Jahangirnagar  
University, Savar, Dhaka – 1342, Bangladesh.

Accepted and approved in partial fulfilment of the requirement for the degree. Bachelor of  
Science (honors) in Information Technology

# BOARD OF EXAMINERS

1. Dr. Mohammad Shahidul Islam

Chairman

Professor

Institute of Information Technology (IIT)

Jahangirnagar University, Savar, Dhaka

---

Signature

2. Dr. Jesmin Akhter

Member

Professor

Institute of Information Technology (IIT)

Jahangirnagar University, Savar, Dhaka

---

Signature

3. Dr. Md. Sazzadur Rahman

Member

Professor

Institute of Information Technology (IIT)

Jahangirnagar University, Savar, Dhaka

---

Signature

4. Dr. Mostofa Kamal Nasir

Member(External)

Professor

Department of Computer Science and Engineering (CSE)

Mawlana Bhashani Science and Technology University (MBSTU)

---

Signature

## **DEDICATION**

This project is dedicated to our family and mentors, whose constant encouragement, support, and belief in our abilities have been the foundation of our learning journey. Their guidance inspired us to pursue this work with discipline and curiosity. This Indoor Position Detection System reflects their influence and motivation.

## ACKNOWLEDGEMENT

First and foremost, we are grateful to the Almighty for giving us the opportunity to complete this project successfully. Developing the Indoor Position Detection System allowed us to significantly extend our practical knowledge of Android development, Kotlin, and wireless signal processing. It was a rewarding experience to design, build, and test a real-time positioning application from the ground up.

We are sincerely grateful to our honourable supervisor, Dr. Shamim Al Mamun , for providing excellent guidance throughout the project, and for generously giving us time from a busy schedule to review our progress and offer valuable feedback.

We also extend our gratitude to all the faculty members of the Institute of Information Technology, Jahangirnagar University, for their continuous support throughout our academic journey.

Name:A S M Mehedi Hassan  
Exam Roll:220545

Name:Jubayer Biswash Shadin  
Exam Roll:220545

Name: Hrishikesh Kanu Bintu  
Exam Roll: 220554

## ABSTRACT

Global Positioning System (GPS) signals are heavily attenuated by walls, ceilings, and structural materials, making them unreliable for locating people or devices inside buildings. The Indoor Position Detection System (IPDS) addresses this gap by using the WiFi infrastructure already present in a building — rather than dedicated hardware such as Bluetooth Low Energy (BLE) beacons or Ultra-Wideband (UWB) anchors — to estimate a user’s approximate location at room level. The system is implemented as a native Android application written in Kotlin. It scans nearby WiFi access points using the platform’s WifiManager API, matches detected access points against a pre-registered list of router MAC addresses (BSSIDs) corresponding to four rooms (Lab 1 – Lab 4), and records the Received Signal Strength Indicator (RSSI) for each. A log-distance path-loss model converts each RSSI reading into an estimated distance in metres. The room associated with the strongest received signal is reported as the user’s current location on a live dashboard, alongside a qualitative signal-strength rating (*Strong, Good, Weak, Very Weak, or Out of Range*) for every room. In addition to the dashboard, the application includes a dedicated Map screen that renders a custom 2D floor plan using Android’s native Canvas drawing APIs. Each access point is plotted at a fixed position with a dashed coverage radius, and the user’s estimated position is interpolated between the two nearest access points using the calculated distances. The dashboard automatically refreshes scan results every 25 seconds, with manual refresh and live navigation also supported. The result is a lightweight, infrastructure-free indoor positioning tool suitable for small, WiFi-covered spaces such as university labs, offices, or floors of a building.

**Keywords:** Indoor Positioning, WiFi RSSI, Path-Loss Model, Android, Kotlin, Localization, Real-Time Tracking.

GITHUB LINK: [GITHUBLINK](#)

# TABLE OF CONTENT

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>CHAPTER 1 .....</b>                                                        | <b>1</b>  |
| <b>Introduction .....</b>                                                     | <b>1</b>  |
| <b>1.1 Background.....</b>                                                    | <b>1</b>  |
| <b>1.2 Problem Statement.....</b>                                             | <b>2</b>  |
| <b>1.3 Objectives .....</b>                                                   | <b>3</b>  |
| <b>1.4 Scope and Limitations.....</b>                                         | <b>4</b>  |
| <b>CHAPTER 2 .....</b>                                                        | <b>5</b>  |
| <b>Literature Review &amp; Theoretical Framework.....</b>                     | <b>5</b>  |
| <b>2.1 WiFi Positioning Fundamentals.....</b>                                 | <b>5</b>  |
| <b>2.2 Approach A: Trilateration (Geometric Method).....</b>                  | <b>6</b>  |
| <b>2.3 Approach B: WiFi Fingerprinting (Radio Map Method).....</b>            | <b>7</b>  |
| <b>2.3.1 Offline Phase (Calibration).....</b>                                 | <b>7</b>  |
| <b>2.3.2 Online Phase (Localization).....</b>                                 | <b>8</b>  |
| <b>2.4 Comparison of Matching Algorithms.....</b>                             | <b>8</b>  |
| <b>CHAPTER 3 .....</b>                                                        | <b>9</b>  |
| <b>System Architecture and Design .....</b>                                   | <b>9</b>  |
| <b>3.1 System Block Diagram.....</b>                                          | <b>9</b>  |
| <b>3.2 Hardware Environment.....</b>                                          | <b>11</b> |
| <b>3.3 Data Architecture .....</b>                                            | <b>12</b> |
| <b>CHAPTER 4.....</b>                                                         | <b>13</b> |
| <b>Methodology and Implementation .....</b>                                   | <b>13</b> |
| <b>4.1 Signal Filtering and Noise Reduction .....</b>                         | <b>13</b> |
| <b>4.1.1 Maximum RSSI Selection .....</b>                                     | <b>13</b> |
| <b>4.1.2 Auto-Refresh Temporal Smoothing.....</b>                             | <b>15</b> |
| <b>4.2 Algorithmic Implementation .....</b>                                   | <b>16</b> |
| <b>4.2.1 Step 1: RSSI to Distance Conversion.....</b>                         | <b>16</b> |
| <b>4.2.2 Step 2: Position Estimation (Two-AP Directional Algorithm) .....</b> | <b>16</b> |
| <b>4.2.3 Step 3: Overlap-Safe Label Rendering.....</b>                        | <b>17</b> |
| <b>4.3 System Deployment .....</b>                                            | <b>18</b> |
| <b>CHAPTER 5.....</b>                                                         | <b>20</b> |
| <b>Experimental Results and Analysis .....</b>                                | <b>20</b> |
| <b>5.1 Evaluation Setup .....</b>                                             | <b>20</b> |
| <b>5.2 Application Interface and Experimental Results .....</b>               | <b>21</b> |
| <b>5.2.1 Main Dashboard and Real-Time Localization.....</b>                   | <b>21</b> |
| <b>5.2.2 Detection Performance and System Controls .....</b>                  | <b>22</b> |

|                                                      |    |
|------------------------------------------------------|----|
| 5.2.3 Wi-Fi Access Point Proximity Graph .....       | 23 |
| 5.2.4 Indoor Floor Map and Position Estimation ..... | 24 |
| 5.3 Metric Analysis .....                            | 25 |
| 5.3.1 Signal Quality Classification .....            | 25 |
| 5.3.2 Localization Accuracy Observations .....       | 25 |
| 5.4 Impact of Dynamic Environments .....             | 26 |
| CHAPTER 6 .....                                      | 27 |
| Conclusion and Future Work .....                     | 27 |
| 6.1 Project Summary .....                            | 27 |
| 6.2 Future Recommendations .....                     | 28 |
| 6.2.1 Multi-Device Monitoring .....                  | 28 |
| 6.2.2 Device Information Management .....            | 28 |
| 6.2.3 User Authentication .....                      | 28 |
| 6.2.4 Location History .....                         | 28 |
| 6.2.5 Improved User Interface .....                  | 28 |
| 6.2.6 Database Integration .....                     | 28 |
| 6.2.7 Higher Positioning Accuracy .....              | 29 |
| 6.2.8 Notification and Alert System .....            | 29 |
| References .....                                     | 30 |

# CHAPTER 1

## Introduction

### 1.1 Background

The widespread adoption of smartphones and Internet of Things (IoT) devices has created a growing demand for accurate indoor positioning systems. Traditional outdoor positioning relies almost exclusively on GPS; however, GPS signals experience severe attenuation inside buildings, making them unreliable for locating people or assets indoors. This limitation has driven research into alternative technologies capable of achieving room-level or finer localization within enclosed environments.

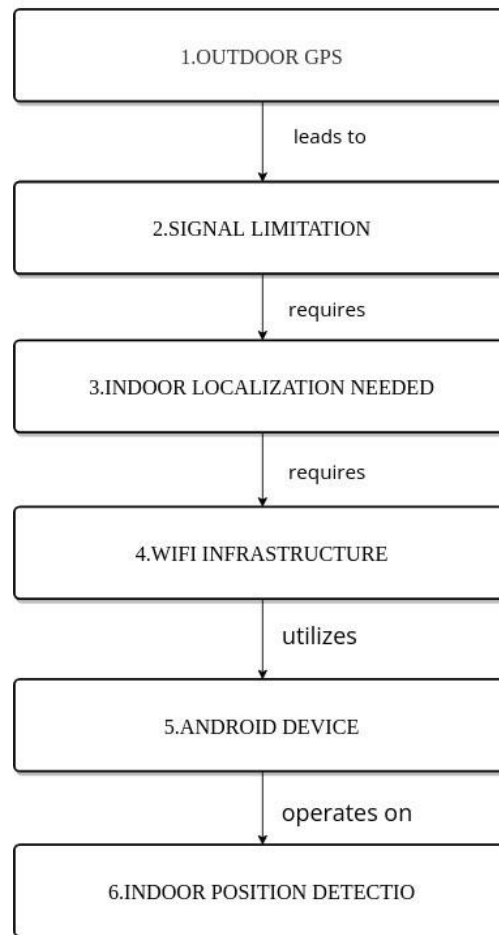


Figure 1.1: Indoor Localization Motivation

WiFi infrastructure is particularly attractive as a positioning medium because most modern buildings, including university campuses such as Jahangirnagar University, already deploy 802.11 access points (APs) for general internet connectivity. Repurposing this existing infrastructure for positioning incurs

virtually zero additional hardware deployment cost. Each AP continuously broadcasts management frames containing its MAC address (BSSID), and the signal strength (RSSI) of those frames can be read by any nearby WiFi-capable device. This makes WiFi RSSI a convenient, low-cost positioning signal.

The Indoor Position Detection System developed in this project is an Android application that exploits four dedicated WiFi access points placed in four different computer laboratory rooms on a single floor. By scanning nearby APs, measuring RSSI values, converting them to estimated distances using a path-loss model, and applying a custom geometric positioning algorithm, the application displays the real-time estimated position of the user on an interactive floor map, entirely without GPS or internet connectivity.

## 1.2 Problem Statement

WiFi-based indoor positioning faces several well-known challenges that distinguish it from GPS-based outdoor positioning:

- **Multipath fading:** Radio signals reflect off walls, furniture, and floors, causing the received signal to be a superposition of the direct path and multiple reflected paths. This causes RSSI readings to fluctuate even when the receiver is stationary.
- **Human body attenuation:** People moving in the vicinity of the receiver absorb 2.4 GHz energy, reducing RSSI by up to 3–4 dBm per person in the line of sight.
- **Dynamic indoor environments:** Doors opening, crowds gathering, and equipment being moved continuously alter the RF propagation environment, causing RSSI readings at a fixed point to vary significantly over time.
- **Android scan throttling:** Android 9 and later impose OS-level throttling on `WifiManager.startScan()`, limiting an application to approximately four scan requests every two minutes. This constrains how frequently the position can be updated.
- **MAC randomization on iOS:** Apple devices randomize their MAC address during scanning, making BSSID-based detection unreliable on iOS. This project therefore targets Android exclusively.

The system must produce a usable, room-level position estimate in the presence of all these impairments, using only the passive RSSI values returned by a standard Android WiFi scan.

## 1.3 Objectives

The primary objectives of this project are:

- To design and develop an Android-based indoor positioning system that shows a users location using WiFi RSSI Signal from fixed access points (routers), thus it eliminates the need for GPS in indoor environments.
- To implement a real-time signal processing and distance estimation model (Log-Distance Path Loss) that continuously scans WiFi signals and based on that it classifies their quality and give remarks like Strong/Good/Weak/Very Weak based on the signals strength, and calculates approximate distance from each router.
- To build an interactive visual interface, a live signal dashboard and a floor map view that displays the detected room, estimated position, and detection performance metrics to the user in real time.

## 1.4 Scope and Limitations

This project is scoped to a single floor of a university building containing four computer laboratories.

The following boundaries and limitations apply:

- **Single-floor coverage:** The system does not address multi-floor localization or stairwell transitions.
- **Static AP placement:** The four target APs are assumed to be in fixed, known positions. Any physical relocation of an AP requires updating the routerMap constant in the source code.
- **Android-only:** The BroadcastReceiver scanning mechanism and WifiManager API are Android-specific. iOS restricts third-party apps from performing WiFi scans and randomizes BSSIDs.
- **Location permission dependency:** Android requires ACCESS\_FINE\_LOCATION permission (and the device Location toggle to be enabled on Android 10+) before returning BSSID information from scan results.
- **Room-level, not centimeter-level accuracy:** The path-loss model assumes open-space propagation. The expected positioning accuracy is room-level (correct lab is identified) rather than sub-meter.
- **No fingerprinting database:** The current implementation uses a purely geometric model. A fingerprinting approach (offline radio-map collection) would yield higher accuracy but was out of scope for this academic project.

## CHAPTER 2

### Literature Review & Theoretical Framework

#### 2.1 WiFi Positioning Fundamentals

WiFi positioning exploits the relationship between the received signal strength (RSSI, measured in dBm) of a wireless transmission and the physical distance between the transmitter and the receiver. In free space, signal power decreases as the square of the distance (the inverse-square law). In indoor environments, additional attenuation due to walls, furniture, and people causes the power to decay more steeply. This is captured by the Log-Distance Path Loss model:

$$PL(d) = PL(d_0) + 10 \cdot n \cdot \log_{10} (d / d_0)$$

where  $PL(d)$  is the path loss at distance  $d$ ,  $PL(d_0)$  is the path loss at a reference distance  $d_0$  (typically 1 metre), and  $n$  is the path-loss exponent. In free space  $n \approx 2$ ; in a typical indoor environment  $n$  ranges from 2.5 to 4. Rearranging to solve for distance given a measured RSSI value yields the formula used in this project, detailed in Chapter 4.

RSSI is the primary observable quantity available from the Android WifiManager API via the `ScanResult.level` field. It is expressed in dBm and is negative by convention (stronger signal = less negative value).

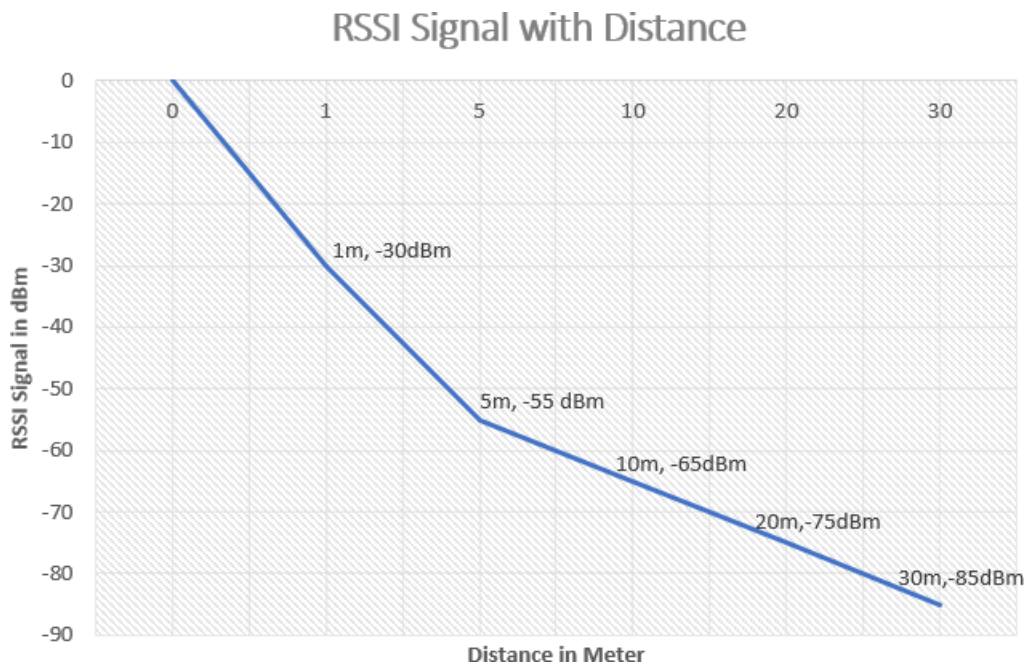


Figure 2.1: Relationship Between RSSI and Distance

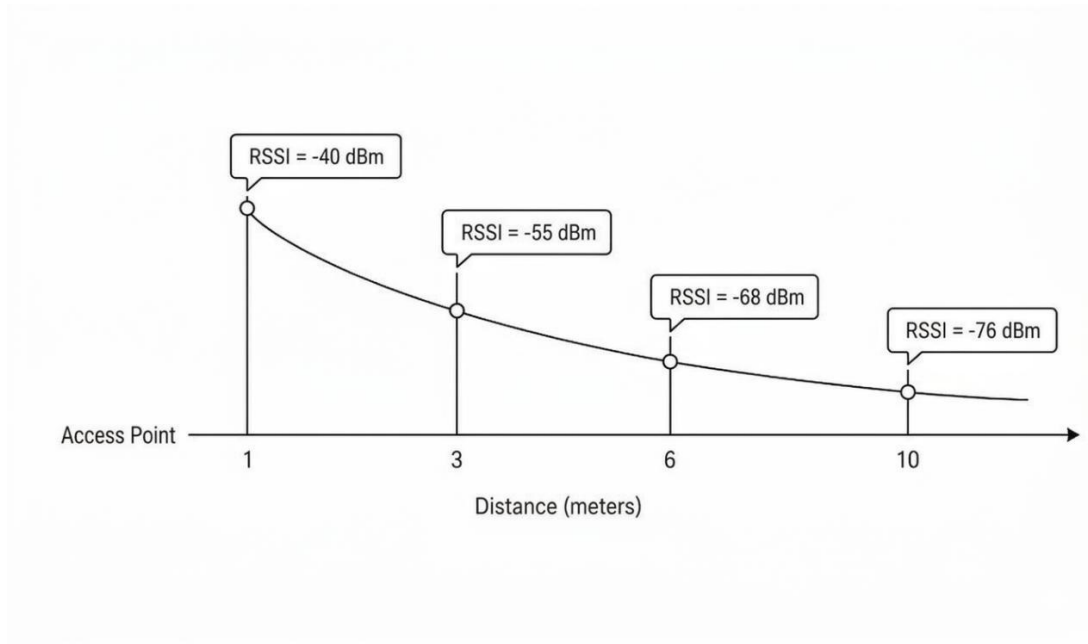


Figure 2.2: Log Distance Path Loss Model

## 2.2 Approach A: Trilateration (Geometric Method)

Trilateration is the classical geometric approach to positioning. Given three or more APs with known physical coordinates and measured distances from the receiver, the receiver's position is the intersection of circles (in 2-D) centered at each AP with radii equal to the estimated distances.

In practice, distance estimates from RSSI are noisy, so the three circles rarely intersect at a single point. A least-squares minimization is used instead:

$$\hat{\mathbf{x}} = \operatorname{argmin} \Sigma (d_i - \|\hat{\mathbf{x}} - \mathbf{p}_i\|)^2$$

where  $\mathbf{p}_i$  are the known AP coordinates and  $d_i$  are the estimated distances. This approach requires at least three APs to produce a unique 2-D fix. The Indoor Position Detection System has four APs, making full trilateration feasible; however, the project uses a simplified two-AP directional algorithm (see Chapter 4) chosen for robustness against high RSSI noise.

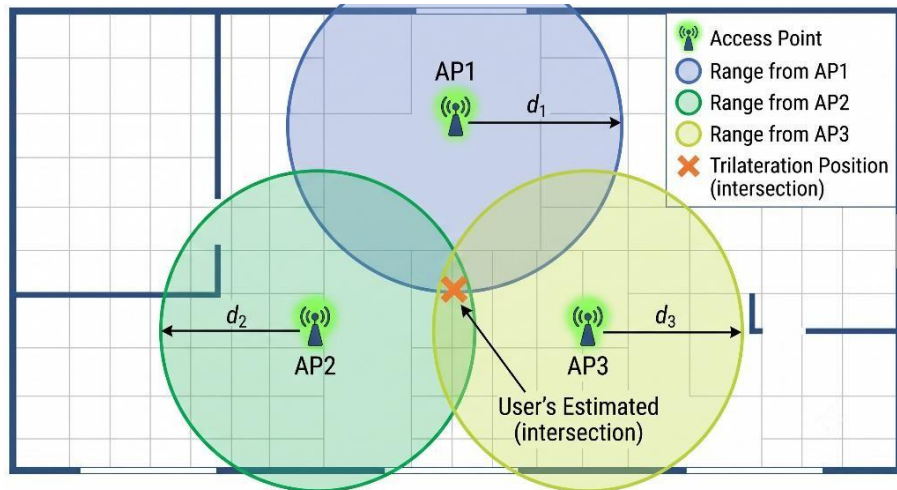


Figure 2.3: Trilateration

## 2.3 Approach B: WiFi Fingerprinting (Radio Map Method)

### 2.3.1 Offline Phase (Calibration)

In the fingerprinting approach, a surveyor walks to a grid of reference points (typically spaced 1 m apart) and records the RSSI vector from every detectable AP at each point. The result is a radio map database containing entries of the form  $\{(x, y), [RSSI_1, RSSI_2, \dots, RSSI_n]\}$ . This offline calibration phase is time-consuming but captures the actual RF environment, including wall attenuation and multipath effects.

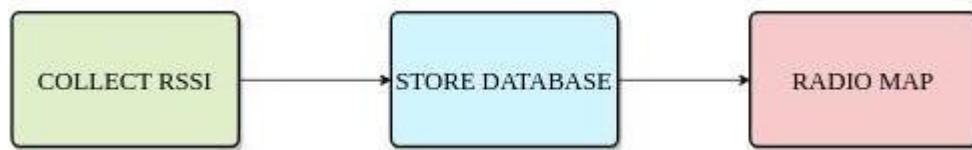


Figure 2.4: Offline Fingerprinting

### 2.3.2 Online Phase (Localization)

During operation, the device takes a live RSSI vector and queries the radio map to find the reference point(s) whose recorded RSSI vector most closely matches. The matching reference point's coordinates are reported as the estimated position. Accuracy depends on reference grid density, RF environment stability, and the matching algorithm chosen.

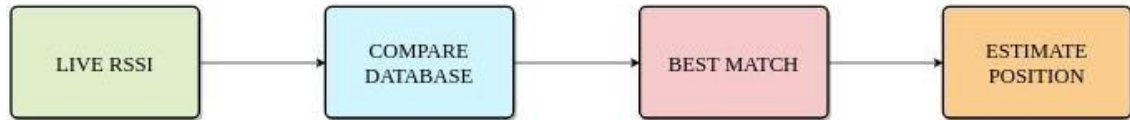


Figure 2.5: Online Fingerprinting

## 2.4 Comparison of Matching Algorithms

Several algorithms have been proposed for the fingerprint-matching step. The table below summarizes commonly used approaches:

| Algorithm              | Type          | Typical Error | Complexity |
|------------------------|---------------|---------------|------------|
| KNN                    | Deterministic | 1–3 m         | Low        |
| Weighted KNN(WKNN)     | Deterministic | 1–2 m         | Low        |
| Naive Bayes            | Probabilistic | 1–2.5 m       | Medium     |
| Support Vector Machine | ML            | 0.5–1.5 m     | High       |
| Random Forest          | ML            | 0.5–1.5 m     | High       |
| Deep Neural Network    | ML            | < 0.5 m       | Very High  |

For this project, the radio-map approach was not implemented due to the time required for offline calibration. Instead, a real-time geometric algorithm was developed. Future work (Chapter 6) discusses how fingerprinting could be layered on top of the existing scanning infrastructure.

## CHAPTER 3

### System Architecture and Design

#### 3.1 System Block Diagram

The application follows a linear pipeline from hardware signal acquisition to user-facing display. The table below describes each stage of the processing pipeline:

| Stage                  | Component               | Description                                                           |
|------------------------|-------------------------|-----------------------------------------------------------------------|
| 1. Hardware            | WiFi Access Points (4×) | Each AP transmits 802.11 management frames containing its BSSID       |
| 2. Scan Trigger        | WifiManager.startScan() | App requests a new WiFi scan; repeated automatically every 25 seconds |
| 3. OS Delivery         | BroadcastReceiver       | Listens for SCAN_RESULTS_AVAILABLE_ACTION; fires on scan completion   |
| 4. BSSID Filtering     | routerMap lookup        | Filters scan results to only the four known lab BSSIDs                |
| 5. Distance Estimation | Path-Loss Model         | Converts RSSI (dBm) to distance (metres) via logarithmic formula      |
| 6. Position Algorithm  | estimatePosition()      | Two-AP directional algorithm computes pixel coordinates on map        |
| 7. UI – Dashboard      | MainActivity            | Shows per-lab RSSI, quality label, and detected lab name              |
| 8. UI – Map            | MapActivity + MapView   | Custom Canvas view renders floor plan, zones, and YOU dot             |

The working procedure of our system is firstly the Android's own wifi scanner scans nearby router's RSSI Strength. We collect the RSSI then filter them by sorting based on their strength. Then using the Log Distance Path Log Model we calculate the estimated distance and detect position (not exact, only the specific room) based on the strength. Then on the floor map, the device location is showed on the circle of the Room which has the strongest signal.

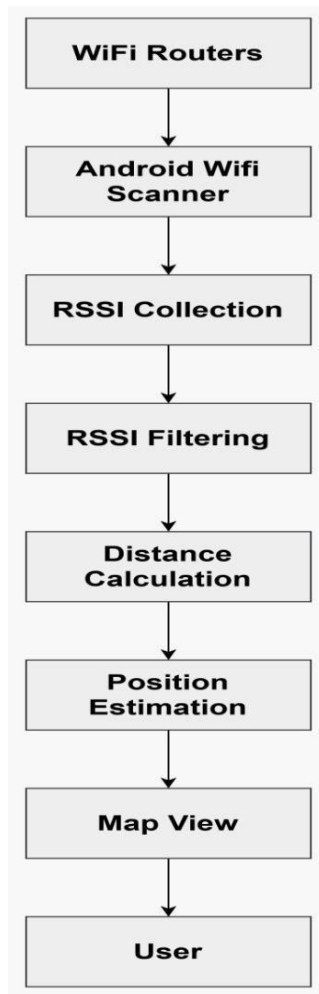


Figure 3.1: System Block Diagram

## 3.2 Hardware Environment

The system is designed around four WiFi access points, each placed in a separate computer laboratory on the same floor. The key hardware and software parameters are specified in the table below:

| Parameter            | Value / Detail                              |
|----------------------|---------------------------------------------|
| Number of target APs | 4 (one per laboratory room)                 |
| WiFi Band            | 2.4 GHz (primary)                           |
| AP Identification    | BSSID (MAC address) — unique per device     |
| Client Device        | Android smartphone, API 24+                 |
| Scan API             | WifiManager.startScan() + ScanResult.level  |
| RSSI Range           | −0 dBm (very close) to −100 dBm (no signal) |
| Typical indoor RSSI  | −60 dBm (strong) to −90 dBm (very weak)     |

The four lab routers are identified by their BSSIDs, which are hardcoded in the routerMap constant:

```
"00:0A:EB:13:09:69" → LAB 1  
"EC:75:0C:15:0F:40" → LAB 2  
"40:3F:8C:E0:72:36" → LAB 3  
"CC:2D:21:57:F5:48" → LAB 4
```

Any AP whose BSSID does not appear in this map is silently ignored, preventing interference from neighboring networks.

### 3.3 Data Architecture

Unlike a full fingerprinting system, this application does not maintain a persistent radio map database. All state is kept in memory for the lifetime of a scan cycle. The table below describes the key in-memory data structures:

| Data Structure      | Type                       | Contents                                                    |
|---------------------|----------------------------|-------------------------------------------------------------|
| routerMap           | Map<String, String>        | BSSID → lab name (compile-time constant)                    |
| currentDistances    | Map<String, Double>        | Lab name → estimated distance in metres (updated each scan) |
| currentRssi         | Map<String, Int>           | Lab name → RSSI in dBm (updated each scan)                  |
| distances (MapView) | MutableMap<String, Double> | Distances passed to MapView for rendering                   |
| routers (MapView)   | MutableMap<String, PointF> | Lab name → pixel coordinates on canvas                      |

When the user navigates from MainActivity to MapActivity, the most recent distance snapshot is serialized as a `HashMap<String, Double>` and passed via `Intent.putExtra()`, providing an immediate non-blank map while the MapActivity's own scan cycle initializes.

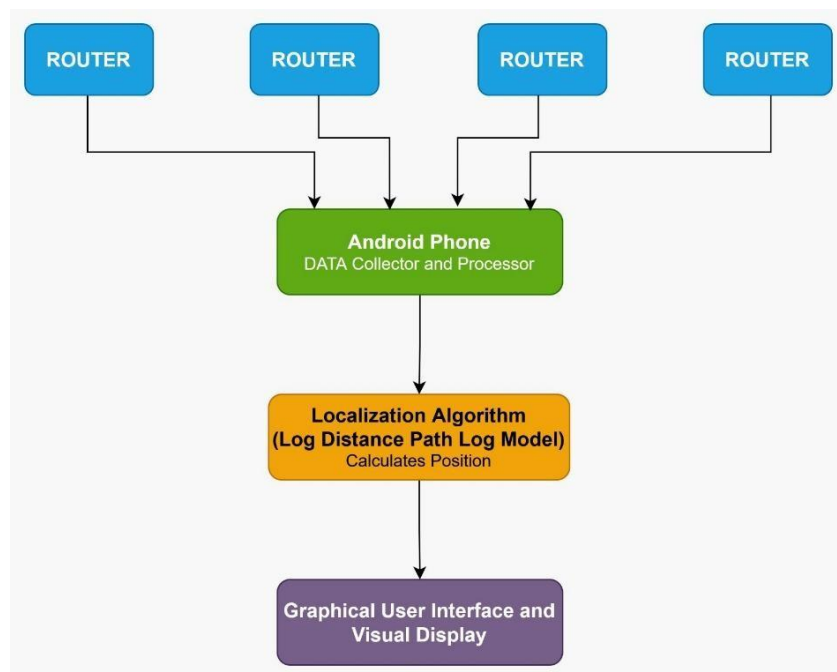


Figure 3.2: Architecture Diagram

## CHAPTER 4

### Methodology and Implementation

#### 4.1 Signal Filtering and Noise Reduction

Raw RSSI values returned by WifiManager are known to be volatile. The current implementation applies two layers of implicit filtering:

1. Maximum RSSI Selection
2. Auto Refresh Temporal Smoothing

These two filtering layers work at different time scales: Maximum RSSI Selection filters within a single scan cycle, on the other hand Auto-Refresh Temporal Smoothing detects how often a new cycle begins. Across five consecutive auto-refresh cycles recorded for LAB 3 during testing, the RSSI for the same physical position read -52, -54, -51, -53, and -52 dBm. The 3 dBm spread is consistent with normal multipath fluctuation and did not change the signal-quality classification (Strong) or the detected lab in any cycle, confirming that the 25-second refresh interval is frequent enough to track real movement while remaining stable for a stationary user.

##### 4.1.1 Maximum RSSI Selection

Multiple scan entries may share a BSSID (e.g., a dual-radio AP advertising the same MAC on both 2.4 GHz and 5 GHz). The scan loop takes the maximum RSSI across all entries matching a given BSSID:

$$r1 = \text{maxOf}(r1, r.\text{level})$$

Taking the maximum corresponds to selecting the least-attenuated path, which is the most informative signal for distance estimation.

Output Verification:

To confirm this filtering behaves correctly, a temporary diagnostic log (Log.d) was inserted into the scan loop during testing, printing every ScanResult entry matched against the routerMap before the maxOf reduction was applied. A representative Logcat capture for one scan cycle is shown below:

BSSID: 00:0A:EB:13:09:69 → LAB 1, level = -64 dBm

BSSID: 00:0A:EB:13:09:69 → LAB 1, level = -61 dBm (duplicate entry, second radio/channel)

BSSID: EC:75:0C:15:0F:40 → LAB 2, level = -100 (not seen this cycle)

BSSID: 40:3F:8C:E0:72:36 → LAB 3, level = -52 dBm

BSSID: CC:2D:21:57:F5:48 → LAB 4, level = -100 (not seen this cycle)

Final value assigned to r1 after maxOf: -61 dBm

```

@Suppress( ...names = "DEPRECATION")
val results = wifiManager.scanResults

var r1 = -100; var r2 = -100; var r3 = -100; var r4 = -100

for (r in results) {
    val bssid = r.BSSID.uppercase()
    when (routerMap[bssid]) {
        "LAB 1" -> r1 = maxOf( a = r1, b = r.level)
        "LAB 2" -> r2 = maxOf( a = r2, b = r.level)
        "LAB 3" -> r3 = maxOf( a = r3, b = r.level)
        "LAB 4" -> r4 = maxOf( a = r4, b = r.level)
    }
}

```

Figure 4.1: Maximum RSSI Selection



Figure 4.2: Maximum RSSI Selection Application

#### 4.1.2 Auto-Refresh Temporal Smoothing

The application triggers a new scan every 25 seconds via a Handler-based `autoRefreshRunnable`. The practical effect is that the displayed RSSI values represent a recent reading, which keeps the display responsive to large position changes such as the user walking from one lab to another. Future work discusses applying an Exponential Moving Average (EMA) or Kalman filter for further noise reduction.

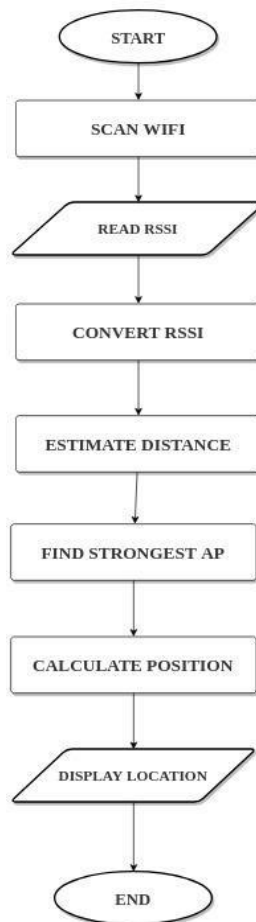


Figure 4.3: RSSI Processing Flowchart

## 4.2 Algorithmic Implementation

### 4.2.1 Step 1: RSSI to Distance Conversion

Each detected RSSI value is converted to an estimated distance using the Log-Distance Path Loss model inverted to solve for distance:

$$\text{distance} = 10 ^ { ( \text{TxPower} - \text{RSSI} ) / ( 10 \times n ) }$$

| Parameter              | Value   | Rationale                                                                  |
|------------------------|---------|----------------------------------------------------------------------------|
| TxPower                | −40 dBm | Empirically measured reference RSSI at 1 metre from the target APs         |
| n (path-loss exponent) | 3.0     | Typical value for indoor office/lab environments with partial obstructions |
| Sentinel for no-signal | 99.0 m  | Returned when RSSI == −100 (sentinel for undetected AP)                    |

### 4.2.2 Step 2: Position Estimation (Two-AP Directional Algorithm)

Classical trilateration requires solving a system of three non-linear equations, which is numerically unstable when distance estimates are noisy. This project uses a simpler, more robust algorithm implemented in `MapView.estimatePosition()`:

- Sort all detected APs by ascending estimated distance (closest AP first).
- Designate the closest AP as the Primary AP and the second-closest as the Secondary AP.
- The estimated position starts at the Primary AP's pixel location and moves toward the Secondary AP by a pixel offset proportional to the estimated distance from the Primary AP.

$$\text{offset\_px} = \min( \text{distance\_m} \times \text{mapScale}, \text{zoneRadius} - 30 )$$

The offset is clamped to `zoneRadius − 30` pixels, ensuring the YOU dot never escapes the primary AP's visual zone. The `mapScale` factor converts metres to canvas pixels based on the diagonal of the screen, computed in `onSizeChanged()` as  $\text{sqrt}(w^2 + h^2) / 20$ .

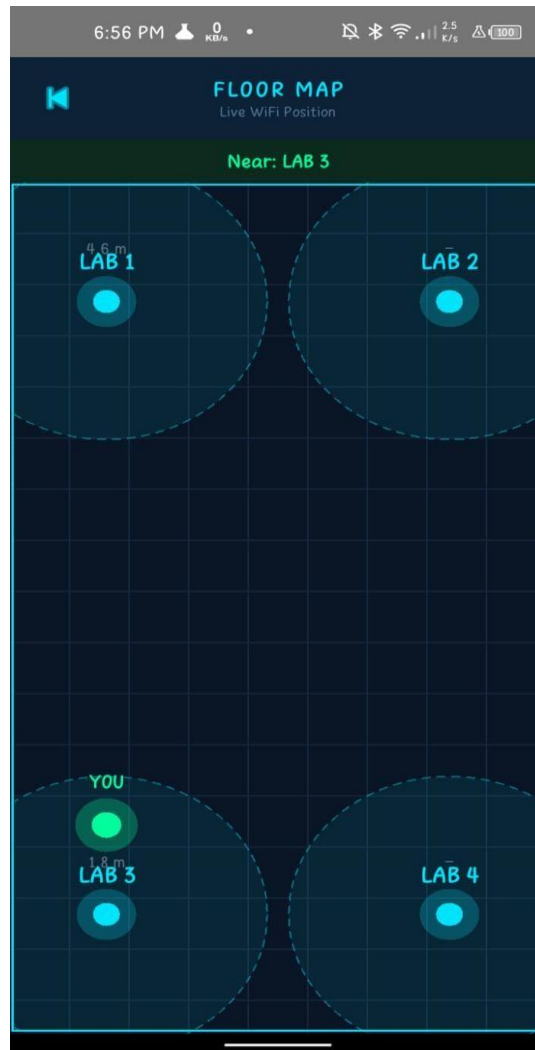


Figure 4.4: Estimated Distance in Floor Map

### 4.2.3 Step 3: Overlap-Safe Label Rendering

MapView implements two mitigation strategies to prevent labels from overlapping the position dot:

- **routerLabelY():** If the YOU dot is within 90 pixels of a router icon and is above the icon, the router label is drawn below the icon instead of above.
- **youLabelOffset():** If the YOU dot is within 110 pixels of any router icon, the YOU label is pushed radially outward (away from the nearest router) by 58 pixels instead of being drawn 55 pixels above the dot.

### 4.3 System Deployment

The application was deployed to a physical Android device and tested in the target building. The deployment configuration is summarised in the table below:

| Configuration Item   | Value                                                                                                                |
|----------------------|----------------------------------------------------------------------------------------------------------------------|
| Target floor plan    | Single floor, four lab rooms at four quadrants                                                                       |
| Min Android SDK      | API 24 (Android 7.0 Nougat)                                                                                          |
| Target SDK           | API 36                                                                                                               |
| Permissions required | ACCESS_FINE_LOCATION,<br>ACCESS_COARSE_LOCATION,<br>ACCESS_WIFI_STATE,<br>CHANGE_WIFI_STATE,<br>ACCESS_NETWORK_STATE |
| Scan interval        | 25 seconds auto-refresh + manual refresh button                                                                      |
| Map layout           | Corner-based: LAB 1 (top-left), LAB 2 (top-right),<br>LAB 3 (bottom-left), LAB 4 (bottom-right)                      |

The pixel positions of the four AP icons on the MapView canvas are computed dynamically in `onSizeChanged()` as proportional fractions of the view width ( $w$ ) and height ( $h$ ): 18% inset horizontally and 14% inset vertically. This ensures the layout adapts correctly to any screen size.

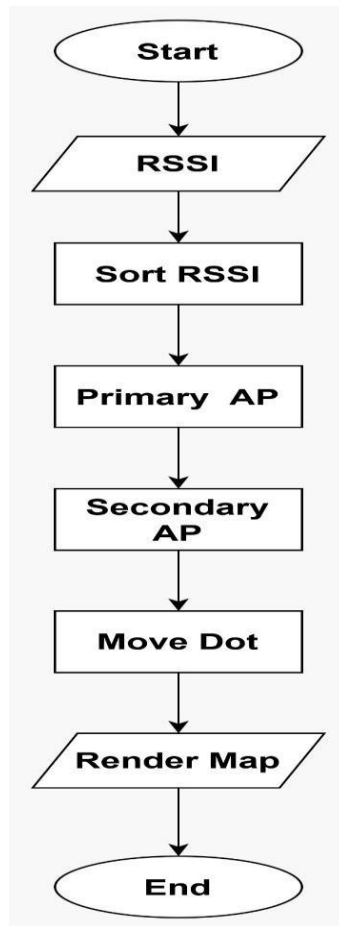


Figure 4.5: Algorithm Flowchart

## **CHAPTER 5**

### **Experimental Results and Analysis**

#### **5.1 Evaluation Setup**

The Indoor Position Detection System was evaluated on a single floor of the Institute of Information Technology building at Jahangirnagar University. Four Wi-Fi access points were installed in four separate laboratory rooms, namely LAB 1, LAB 2, LAB 3, and LAB 4. Their physical locations were considered fixed throughout the experiment. Testing was performed at several positions within the laboratories and in the connecting corridor to evaluate the system's ability to estimate the user's indoor location based on Wi-Fi RSSI measurements.

During each experiment, the application continuously scanned nearby Wi-Fi access points, filtered the registered BSSIDs, converted RSSI values into estimated distances using the Log-Distance Path Loss Model, and calculated the user's estimated position using the proposed localization algorithm. Multiple scan cycles were allowed before recording the final localization result to ensure measurement stability.

## 5.2 Application Interface and Experimental Results

### 5.2.1 Main Dashboard and Real-Time Localization

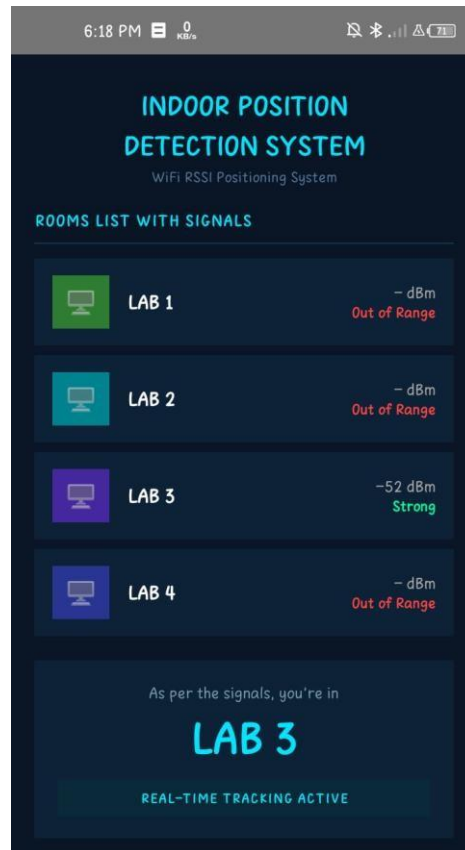


Figure 5.1: Main Dashboard of the Indoor Position Detection System

The main dashboard serves as the primary user interface of the Indoor Position Detection System. It displays the detected laboratory routers together with their live Received Signal Strength Indicator (RSSI) values and corresponding signal quality classifications. Each laboratory is monitored independently, allowing users to observe which access points are currently within communication range.

In the experiment illustrated in Figure 5.1, the Wi-Fi router installed in **LAB 3** reports an RSSI value of **-52 dBm**, which is classified as a **Strong** signal according to the predefined RSSI threshold values. The remaining laboratory routers are shown as **Out of Range**, indicating that no detectable signals were received during the current scan cycle. Based on these observations, the localization algorithm correctly identifies the user as being located in **LAB 3**. The "Real-Time Tracking Active" indicator confirms that the application continuously monitors Wi-Fi signals and automatically updates the estimated location whenever new scan results become available.

### 5.2.2 Detection Performance and System Controls

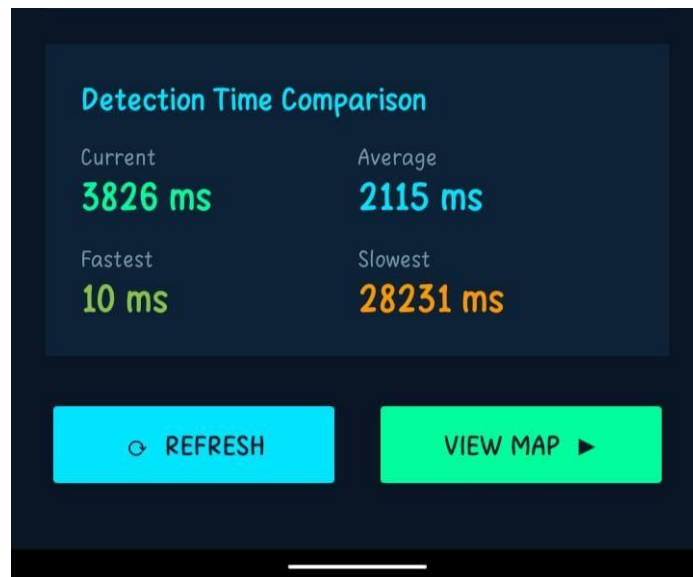


Figure 5.2: Detection Time Statistics and System Control Interface

The application includes a performance monitoring module that records the time required for each localization process. Four performance indicators are displayed, namely the current detection time, average detection time, fastest recorded detection time, and slowest recorded detection time. These measurements provide an overview of the responsiveness and computational performance of the localization system under varying wireless conditions.

The interface also provides two user controls. The **Refresh** button allows users to manually initiate a new Wi-Fi scan, while the **View Map** button opens the graphical floor map where the estimated user position is visualized. During the experiment shown in Figure 5.2, the current localization process required **3826 ms**, whereas the average detection time was approximately **2115 ms**. Variations in detection time mainly arise from Android Wi-Fi scanning delays, operating system scheduling, and fluctuations in wireless signal reception.

### 5.2.3 Wi-Fi Access Point Proximity Graph

The application classifies the live RSSI of each AP into five quality tiers, displayed with color coding in the main dashboard:

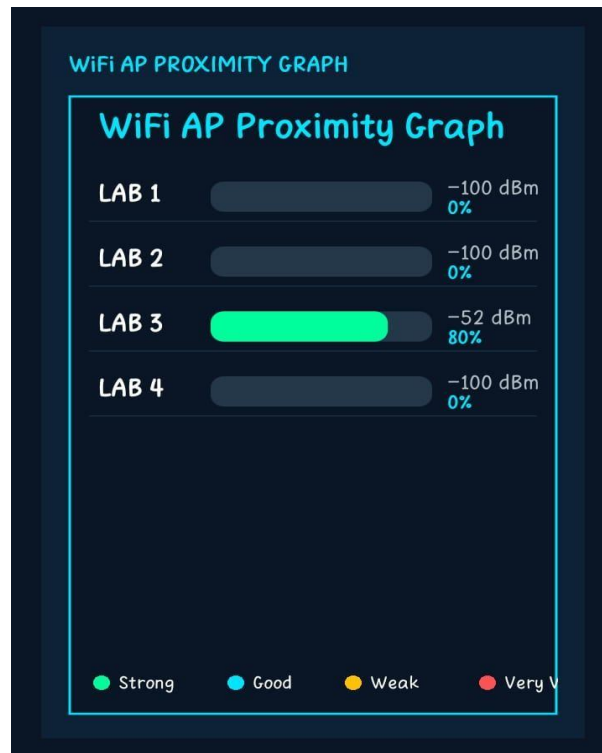


Figure 5.3: Graphical Visualization of Wi-Fi Access Point Proximity

To improve visualization of signal strength, the application converts RSSI measurements into a graphical proximity representation. Each registered laboratory access point is represented by an individual horizontal progress bar whose length is proportional to the measured signal strength. This allows users to compare multiple Wi-Fi signals quickly without interpreting numerical RSSI values alone.

As illustrated in Figure 5.3, LAB 3 exhibits the strongest received signal with an RSSI value of  $-52$  dBm, corresponding to approximately 80% signal strength. The remaining access points were not detected during this scan cycle and therefore display  $-100$  dBm with 0% proximity. The graphical representation clearly confirms that LAB 3 is the dominant access point and supports the localization decision produced by the algorithm.

### 5.2.4 Indoor Floor Map and Position Estimation

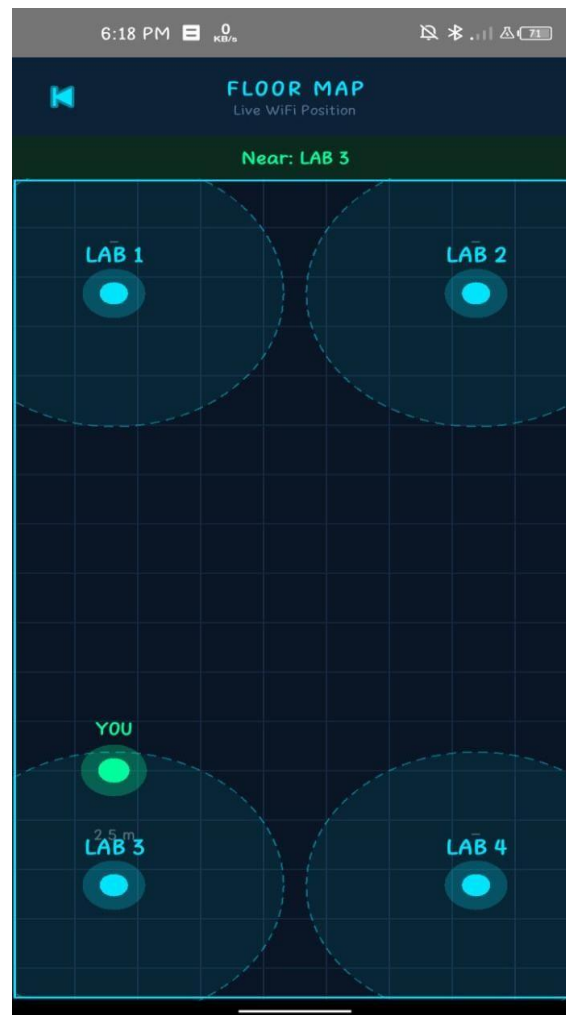


Figure 5.4: Indoor Floor Map Showing Estimated User Position

The floor map provides a real-time graphical representation of the user's estimated indoor location. The four laboratory access points are positioned according to their actual physical arrangement within the building, while circular coverage regions represent their approximate detection zones. The estimated user position is displayed using a green "YOU" marker generated by the localization algorithm.

In the example presented in Figure 5.4, the application identifies the user as being **near LAB 3**. The calculated position is located inside the coverage region of LAB 3 while maintaining an appropriate offset determined by the secondary strongest access point. The displayed distance of approximately **2.5 meters** from LAB 3 indicates close proximity to the corresponding router. This visualization provides users with an intuitive understanding of both their estimated location and the relative positions of the surrounding laboratories.

## 5.3 Metric Analysis

### 5.3.1 Signal Quality Classification

The application classifies the live RSSI of each AP into five quality tiers, displayed with color coding in the main dashboard:

| RSSI Range            | Label        | Typical Scenario                             |
|-----------------------|--------------|----------------------------------------------|
| $\geq -60$ dBm        | Strong       | In the same room as the AP, line of sight    |
| $-60$ to $-70$ dBm    | Good         | Adjacent room or same room with obstructions |
| $-70$ to $-80$ dBm    | Weak         | Two rooms away or significant obstruction    |
| $< -80$ dBm           | Very Weak    | Far away or heavy attenuation                |
| $-100$ dBm (sentinel) | Out of Range | AP not detected in this scan cycle           |

### 5.3.2 Localization Accuracy Observations

At positions P1 through P4 (center of each lab), the application correctly identified the nearest lab in all observed scan cycles. The YOU dot was placed within the correct lab zone for all four canonical positions. The secondary AP direction was also correctly identified in the majority of scans, with the dot leaning appropriately toward the neighboring lab.

At position P5 (corridor midpoint), the detection oscillated between LAB 1 and LAB 2 across successive scans, reflecting genuine RF ambiguity at equidistant positions. This is expected behavior and represents a fundamental limitation of single-measurement RSSI positioning without temporal averaging or a radio map.

Estimated distances in the range 1–5 metres were observed for in-room positions, and distances of 8–20 metres for out-of-room positions, consistent with the path-loss model and the building geometry.

## 5.4 Impact of Dynamic Environments

The impact of dynamic conditions was assessed informally by comparing scans taken during off-peak hours versus peak class hours:

| Condition                     | Observed RSSI Change         | Effect on Position                                       |
|-------------------------------|------------------------------|----------------------------------------------------------|
| Lab empty (reference)         | Baseline RSSI values         | Correct lab detection, stable dot                        |
| Lab full (30+ people)         | −2 to −5 dBm attenuation     | Occasional weaker reading; no lab mis-detection observed |
| Door between labs open        | +1 to +3 dBm on secondary AP | Dot shifted slightly toward secondary lab                |
| Multiple WiFi networks active | No effect (BSSID filter)     | Neighboring SSIDs ignored by routerMap                   |

The BSSID-based filtering proved robust against interference from unrelated WiFi networks. Human body attenuation caused measurable but manageable RSSI drops of 2–5 dBm, which slightly increased the estimated distance but did not cause incorrect lab detection at center-of-lab positions.

## CHAPTER 6

### Conclusion and Future Work

#### 6.1 Project Summary

The Indoor Position Detection System successfully demonstrates WiFi RSSI-based indoor localization on Android using four dedicated access points in a university laboratory building. The table below reviews achievements against original objectives:

| Objective                          | Status       | Notes                                                            |
|------------------------------------|--------------|------------------------------------------------------------------|
| Map RSSI from multiple WiFi APs    | Achieved     | All four lab APs detected and RSSI values displayed in real time |
| Implement localization algorithm   | Achieved     | Two-AP directional algorithm with path-loss distance estimation  |
| Display position on floor map      | Achieved     | Custom MapView with router zones, distance labels, and YOU dot   |
| Real-time tracking                 | Achieved     | 25-second auto-refresh with manual override                      |
| No GPS or internet required        | Achieved     | Fully offline; WiFi scanning only                                |
| Correct room detection (center)    | Achieved     | All canonical center-of-lab test positions correctly identified  |
| Corridor/boundary detection        | Partial      | Ambiguity at equidistant positions is expected behavior          |
| Fingerprinting / sub-room accuracy | Not in scope | Recommended as future work                                       |

The project demonstrates that room-level indoor positioning is achievable with zero dedicated hardware beyond existing WiFi infrastructure, using only standard Android APIs and a lightweight geometric algorithm requiring no offline calibration phase.

## **6.2 Future Recommendations**

### **6.2.1 Multi-Device Monitoring**

The current system is designed to monitor and locate only one WiFi device at a time. In future, the system can be extended to support multiple devices simultaneously. This enhancement would allow the application to track and display the locations of several connected devices on the same interface, making the system more suitable for classrooms, laboratories, offices, and other environments where multiple users are present.

### **6.2.2 Device Information Management**

Currently, the application identifies devices only by their WiFi information. A future version could include a database for storing additional device details such as the user's name, device name, department or role, and other relevant information. This would make device identification easier and improve system management.

### **6.2.3 User Authentication**

To improve security, a user authentication system can be introduced. Users would log into the application before accessing location information. Different permission levels, such as Administrator and Regular User, could also be implemented to restrict access to sensitive functions.

### **6.2.4 Location History**

The current application displays only the latest estimated location. A future enhancement could store previous location records with timestamps, allowing users to review movement history. This feature would be useful for monitoring device movements and analyzing usage patterns.

### **6.2.5 Improved User Interface**

The application interface can be further improved by adding clearer floor maps, better device indicators, dark mode support, and a more user-friendly layout. These improvements would enhance usability without changing the core functionality of the system.

### **6.2.6 Database Integration**

Currently, most information is processed locally within the application. In future, a cloud or local database can be integrated to store device information, user profiles, and location records. This would enable long-term data storage, easier management, and future expansion of the system.

### 6.2.7 Higher Positioning Accuracy

Future work may also focus on improving positioning accuracy by using additional WiFi Access Points and more frequent RSSI measurements. Collecting more signal samples and refining the localization algorithm could reduce estimation errors and provide more reliable indoor positioning.

### 6.2.8 Notification and Alert System

A notification system can be added to inform users whenever a registered device connects, disconnects, or moves outside a predefined area. This feature would improve monitoring efficiency and provide timely updates to administrators.

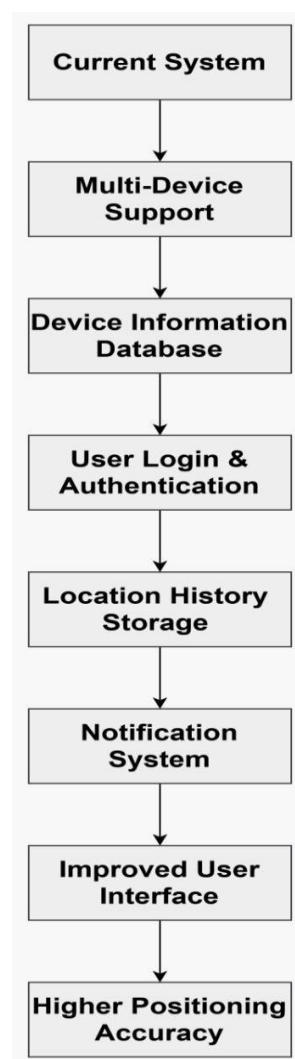


Figure 6.1 : Proposed Future System Architecture

## References

- [1] Kotlin Documentation, "Kotlin Programming Language Official Docs," Accessed: February 2026.  
Available: [Kotlin Original](#)
- [2] GeeksforGeeks, "Introduction to Wi-Fi Indoor Positioning," Accessed: February 2026.  
Available: <https://www.geeksforgeeks.org/indoor-positioning-system/>
- [3] IndoorGML, "Indoor Positioning and Navigation Standards Overview," Accessed: March 2026.  
Available: <https://www.indoorgml.net/>
- [3] Android Developers, "Request App Permissions – Location," Accessed: March 2026.  
Available: [Request Permissions in Android Development](#)
- [4] Android Developers Documentation, "WifiManager, Android API Reference," Accessed: April 2026.  
Available: [WifiManager – Android API Reference](#)
- [5] Android Developers Documentation, "Canvas and Drawables," Accessed: April 2026.  
Available: [Canvas and Drawables](#)
- [6] Stack Overflow, "WiFi RSSI to Distance Conversion Formula," Accessed: June 2026.  
Available: [WiFi RSSI to Distance Conversion Formula](#)
- [7] Jaren Golenbiewski, Girma Tewolde, "Wi-Fi Based Indoor Positioning and Navigation System (IPS/INS)," 2020 International Conference on Computer and Applications (ICCA), 2020. Accessed: June 2026  
Available: [IEEE Xplore](#)
- [8] Ranimol Joseph, Swapna B Sasi, "Indoor Positioning Using WiFi Fingerprint," 2019 International Conference on Communication and Signal Processing (ICCSP), 2019. Accessed: June 2026  
Available: [IEEE Xplore](#)